Lab 6-2
SystemVerilog Operator

Prof. Chih-Cheng Tseng (曾志成)

cctseng@mail.ntou.edu.tw

Contents of this slide are provided by NTOU VLSI Lab601

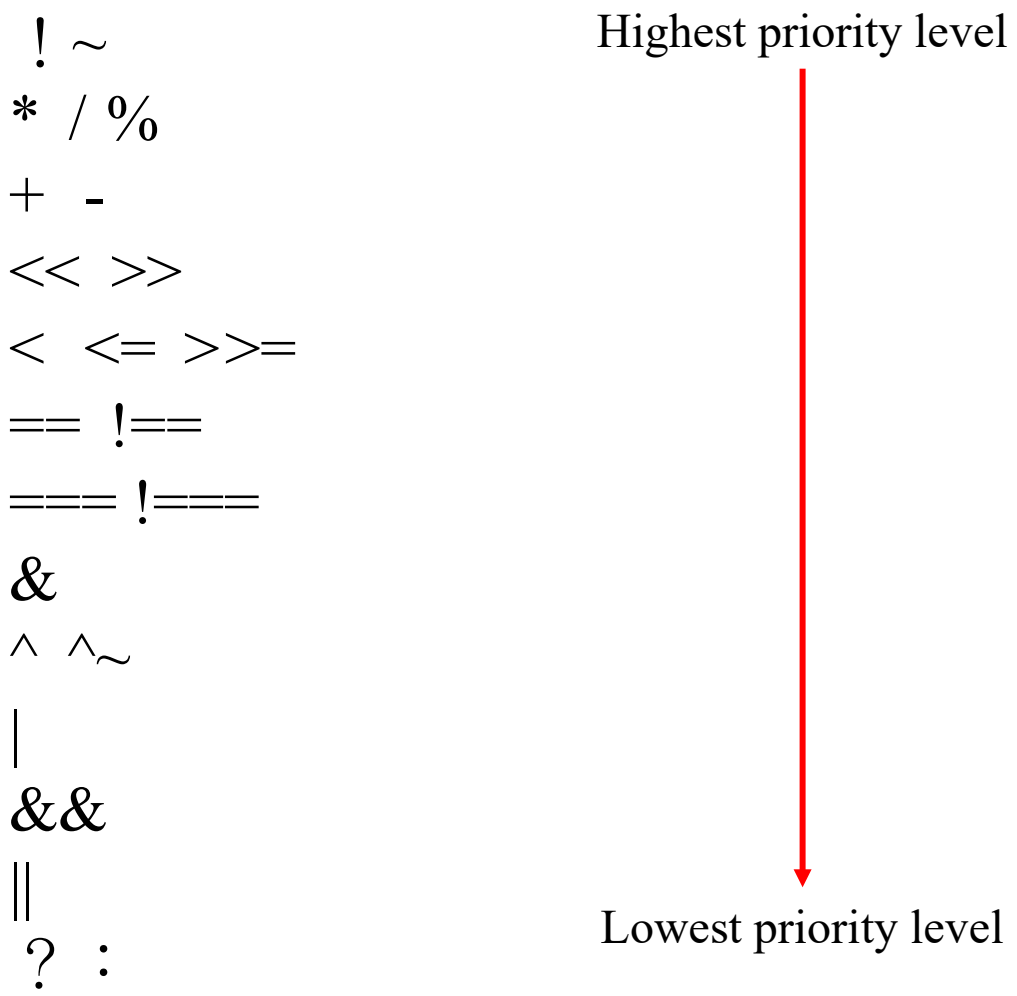
Objectives

- Types of Operators
- Precedence Order
- Relational Operators
- Equality Operators
- Shift Operators
- Concatenation Operators
- Reduction Operators

Types of Operators

- The SystemVerilog HDL language operators are very broad, and can be roughly divided into the following categories based on their functionality:
  1. Arithmetic Operators: +, -, *, /, % (modulo)
  2. Assignment Operators: =, <=
  3. Relational Operators: >, <, >=, <=, ==. !=
  4. Logical Operators: && (AND), || (OR), ! (NOT)
  5. Conditional Operator: ?:
  6. Bitwise Operators: ~ (NOT), | (OR), ^ (XOR), & (AND), ~^ (XNOR)
  7. Shift Operators: << (shift to the left), >> (shift to the right)
  8. Concatenation Operator: {}
  9. Others

Priority Order



Relational Operators

- “<”, “>”, “<=”, “>=”
  - If the relational operation is false, the return value is 0; if the relation is true, the return value is 1.
- The priority level of relational operators is lower than that of arithmetic operators.
  - Example:
    - “ a < size – 1 ” equals to “ a < (size – 1) ”
    - “ size – 1 < a ” is not equal to “ size – (1 < a) ”

Equality Operators

- “==”, “!=” , “===”, “!==”:
  - There should be no spaces between these symbols.
- The difference between logical equality operators and case equality operators:
  - == and != are called logical equality operators, and their result is determined by the values of the two operands. Since the operands may be x or z, the result could be x.
  - === and !== are commonly used in case expressions, also called case equality operators. Their results are only 0 and 1. If the operands contain x or z, the operands must be identical for the result to be 1; otherwise, the result will be 0."

Shift Operators

- "<<", ">>"
  - a>>n where "a" represents the operand to be shifted, and "n" represents how many positions to shift. In both of these shift operations, 0 is used to fill the vacated positions.
  - If the operand has a defined width, after the shift operation, the operand changes, but its width remains unchanged.
- << Left Shift // => Multiply by two
- >> Right Shift // => Divide by two
  - Example: Let b = 4'b1010;
    - a = b << 1; // b left shift by one bit = 5'b10100 = 5'd20
    - a = b >> 1; // b right shift by one bit = 5'b00101 = 5'd5

Concatenation Operators

- {signal1's certain bits, signal2's certain bits, ... Signal*n*'s certain bits}
  - This expression lists certain bits of signals, separated by commas and enclosed in curly braces, representing a composite signal.
- In a bit concatenation expression, multi-bit signals without specified bit widths are not allowed.
- Example: Assume b is a multi-bit signal
  - {a, b[3:0], w, 3'b101} // equals to {a, b[3], b[2], b[1], b[0], w, 1'b1, 1'b0, 1'b1}
  - {4{w}} // equals to {w, w, w, w}
  - {c, {3{a, c}}} // equals to {c, a, c, a, c, a, c}.
    // 3 and 4 must be constant.

Reduction Operators

- This is a unary operator, which includes ~, |, ^, &, ^~.
- The operation rules are similar to bitwise operations, but the operation is performed on each bit of the operand individually.
- The final result of the operation is a single-bit binary number.
- Example:

```
reg [3:0] B;
reg C;
assign C = &B; // " C = &B " equals to "C=((B[0]&B[1])&B[2])&B[3]"
```